Finding Domain Boundaries

A Practical Framework for Bank Core Modernization

How banks can identify the right seams to break apart legacy cores, without breaking the business ?

Architecture Review Committee | Enterprise Technology

Placida Fernando 09/04/2026

Executive Summary

This is an answer to a common question I get from some of the banks and credit unions I have worked in the past

Most banks do not fail at modernization because they picked the wrong cloud, the wrong core vendor, or the wrong microservices framework. They fail because they never correctly identified **where one part of the business ends and another begins** and then built new architecture on top of the old, wrong boundaries.

This paper lays out a practical, repeatable framework for finding **domain boundaries** in a bank: the natural seams in the business - deposits, lending, payments, customer servicing, fraud and risk, that should map to independent, loosely-coupled systems. It draws on Domain-Driven Design (DDD), the BIAN (Banking Industry Architecture Network) reference model, and lessons from real core-modernization and "hollow-out" engagements at regional and national banks.

The goal isn't a diagram for its own sake. A correctly drawn domain boundary is what lets a bank:

- Replace one piece of a 30-year-old core (say, deposit servicing) without touching lending, cards, or the general ledger
- Stand up new digital banking or LOS platforms that integrate cleanly instead of accreting point-to-point spaghetti
- Let vendor and in-house teams own clear, non-overlapping pieces of the system
- Negotiate from strength with core vendors, instead of being told what's technically possible

Why Domain Boundaries Are Hard in Banking ?

Modernization often fails when the bank identifies the wrong business boundaries.

The framework combines Domain-Driven Design, BIAN, Event Storming, context mapping, data ownership, team topology, and sequencing.

A concrete banking example: the word **"account"** means something different to:

- **Deposit operations** (a ledger of balances and holds)
- **Digital banking** (a thing a customer views, renames, and links to Zelle/P2P)
- **Fraud & risk** (a bundle of behavioral signals and risk scores)
- **Marketing/CRM** (a relationship with cross-sell potential)

Trying to build one shared "Account" data model that serves all four teams equally well is exactly how banks end up with a 400-column monster table and a change-management process that takes six months to add a field. Each of those four is a different bounded context, and each deserves its own model of "account," synchronized through **events**, not a shared database.

A well-run domain-boundary exercise produces three artifacts:

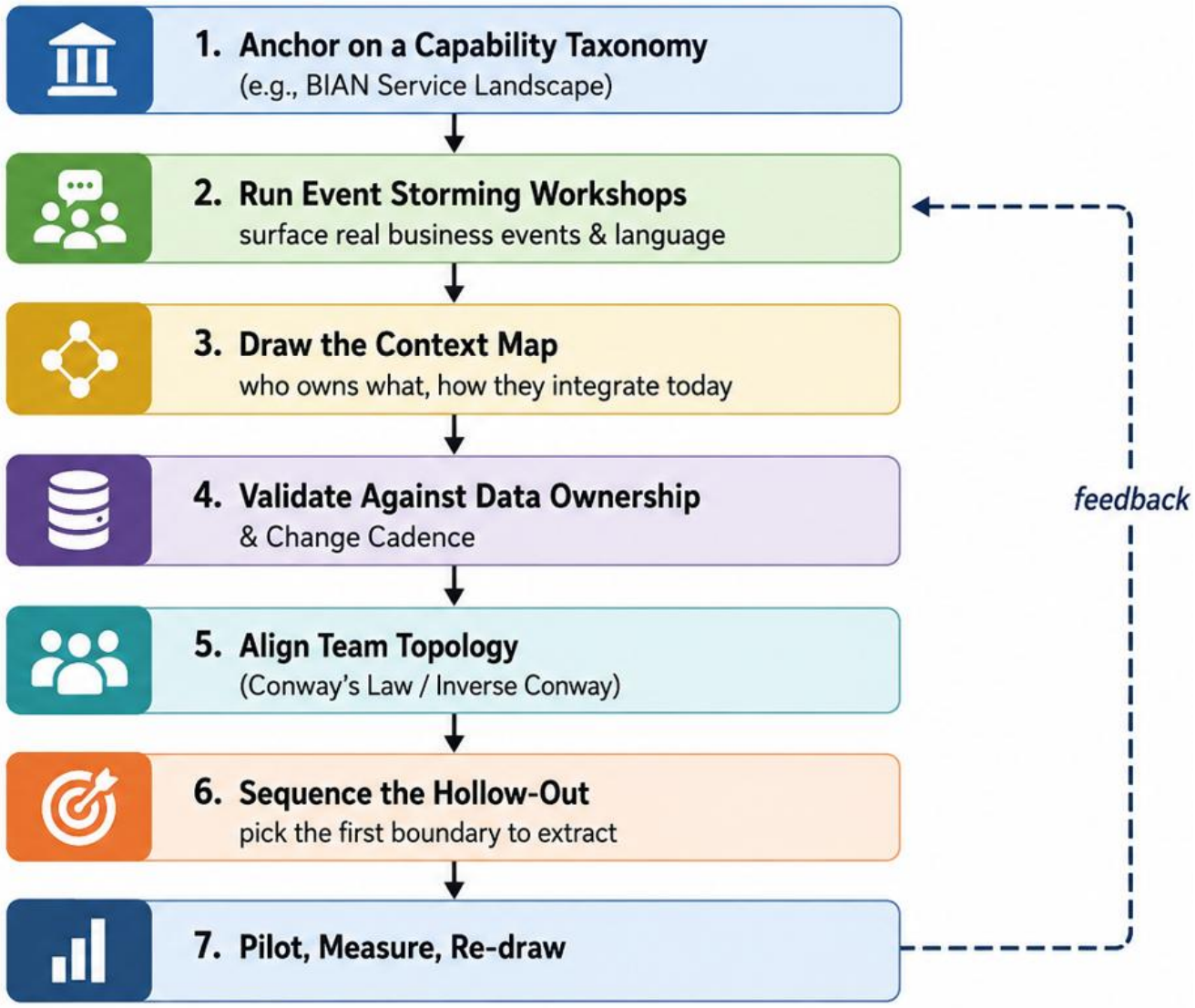
- **A domain map** , the business capabilities (deposits, lending, payments, servicing, risk, etc.) and how they relate
- **A context map** , which systems/teams own which bounded contexts today, and what integration pattern connects them (see Section 4)
- **A sequencing plan** , which boundary to peel off first, second, third, when modernizing

What is bounded Context ?

- A bounded context is a boundary within which a business model, vocabulary, and set of rules are consistent.
- The same term can have different meanings across contexts.
- Example: 'Account' can mean a ledger of balances and holds in deposit operations, a customer-facing object in digital banking, behavioral signals in fraud, or a relationship in marketing.
- Do not force every team to share one canonical model when the business meanings and rules differ.

Practical Discovery Framework Flowchart

A step-by-step approach to identify, validate, and sequence domain boundaries for core modernization



Step 1) How to use BIAN Properly ?

- Use BIAN as an industry reference model and checklist, not as a blueprint for the target architecture.
- Treat BIAN Service Domains as candidate bounded contexts, not confirmed boundaries.
- Use BIAN vocabulary to expose differences in terminology and meaning.
- Reuse useful semantic operations and contract concepts as a starting point for APIs and events.
- Validate BIAN candidates with the actual business, data ownership, rules, change cadence, and organizational model.

What BIAN actually gives you. BIAN organizes the whole of banking into a hierarchy: **Business Areas** (e.g., Customer Management, Product Fulfilment & Operations, Sales & Servicing) contain **Business Domains** (e.g., Deposit Accounts, Consumer Loans, Payments), which are made up of roughly 300+ **Service Domains** , granular, semantically precise units like *Current Account*, *Consumer Loan*, *Payment Order*, *Party Reference Data Management*, *Card Authorization*, and *Fraud Response*. Each Service Domain comes pre-defined with a control record, a set of business scenarios, and a set of standard service operations (Initiate, Update, Retrieve, Control, Exchange). That last part matters: BIAN isn't just a taxonomy of names, it's a semantic API specification , which means it can double as a starting point for the **Published Language** and **Open Host Service** contracts described in Step 3, not just a slide of boxes

Step 2) Event Storming: Find the Real Seams

- Bring business SMEs together with architects from deposits, lending, servicing, fraud, and digital banking.
- Map business events such as Account Opened, Payment Returned, Loan Charged Off, and Dispute Filed.
- Look for changes in language, contested ownership, and events with many downstream consumers.
- Business event language often reveals boundaries more accurately than legacy code or database tables.

Step 3) Context Mapping Patterns

Pattern	When to use it	Banking example
Anti-Corruption Layer (ACL)	Protecting a new domain model from a legacy core's data shapes	A new deposit-servicing microservice translates IBS's legacy field layouts into a clean domain model
Open Host Service	One team exposes a stable, well-documented API for many consumers	A "Customer 360" service exposing party data to lending, servicing, and fraud alike
Conformist	A downstream team just accepts the upstream model as-is (usually temporary)	A reporting team consuming the core's data model unchanged, pending future decoupling
Shared Kernel	Two closely related teams deliberately share a small, jointly-owned model	Core deposits and core lending sharing a common "Party" and "Product" schema
Published Language	A common, versioned event schema used across many contexts	A bank-wide "Transaction Posted" event schema consumed by fraud, GL, and digital banking

Step 3) Context Mapping Patterns cont. : Banking Example

Worked example: from BIAN Service Domain to bounded context.

A representative slice of this exercise, applied to a legacy-core deposit and lending platform, looks like this:

BIAN Service Domain(s)	Business Capability	Target Bounded Context	Recommended Pattern
Current Account, Savings Account	Deposit account servicing	Deposit Servicing (stays on legacy core near-term)	ACL in front of the core; core remains system of record
Party Reference Data Management	Customer identity & relationship data	Party / Customer 360 (extracted early)	Open Host Service , single authoritative API consumed by lending, servicing, fraud
Consumer Loan, Loan Origination	Lending origination & underwriting	Loan Origination (owned by LOS platform, not core)	Published Language event contract back to the core for booking/servicing handoff
Card Authorization, Fraud Response	Real-time transaction risk	Fraud & Risk Decisioning (independent, low-latency service)	Open Host Service with its own event stream; explicitly not a shared kernel with deposits
Payment Order, Payment Execution	Payments initiation & clearing	Payments (extracted mid-sequence)	ACL to legacy core for GL posting; Published Language for payment status events

Two things tend to jump out once a bank does this mapping honestly. First, **Party/Customer data is almost always the highest-value, earliest extraction candidate** , nearly every other domain depends on it, and legacy cores rarely model it cleanly, so getting one authoritative Party service in place early de-risks everything downstream. Second, **Fraud & Risk should almost never be a shared kernel with core deposit or lending data** , it needs to evolve and retrain independently, on its own cadence, which is exactly the "cadence test" from Step 4 flagging a real boundary.

Step 4) Validate the Boundary

Ownership test: Can one team or system be authoritative for the data without repeated cross-domain approval?

Cadence test: Do the candidate domains change for different business reasons and on different schedules?

A domain that has independent rules, ownership, and change cadence is a strong candidate for a separate bounded context.

Step 5) Align Team Topology

A domain boundary that isn't backed by a matching team boundary will erode within a year. Use the **Inverse**

Conway Maneuver deliberately: once you've drawn the target context map, consider restructuring (or at least

clarifying decision rights within) teams to mirror it , one team per bounded context, with a clear API/event

contract to its neighbors, rather than one team touching many contexts inside a shared core.

Not every boundary should be extracted first. Prioritize using two axes: business value of independent change vs. technical difficulty of extraction. In most core-modernization programs, the highest-value, lowest-risk first move is a domain that:

- Changes frequently for competitive/product reasons (a strong pull toward independence)
- Has relatively few, well-understood integration points with the rest of the core
- Doesn't sit on the critical path of end-of-day batch or GL posting (lower blast radius if something goes wrong)

Digital banking and deposit product configuration are frequently good first candidates; general ledger and core-posting logic are usually last, precisely because everything else depends on them

Step 6) Sequence the Hollow-out

Not every boundary should be extracted first. Prioritize using two axes: business value of independent change vs. technical difficulty of extraction. In most core-modernization programs, the highest-value, lowest-risk first move is a domain that:

- Changes frequently for competitive/product reasons (a strong pull toward independence)
- Has relatively few, well-understood integration points with the rest of the core
- Doesn't sit on the critical path of end-of-day batch or GL posting (lower blast radius if something goes wrong)

Digital banking and deposit product configuration are frequently good first candidates; general ledger and core-posting logic are usually last, precisely because everything else depends on them

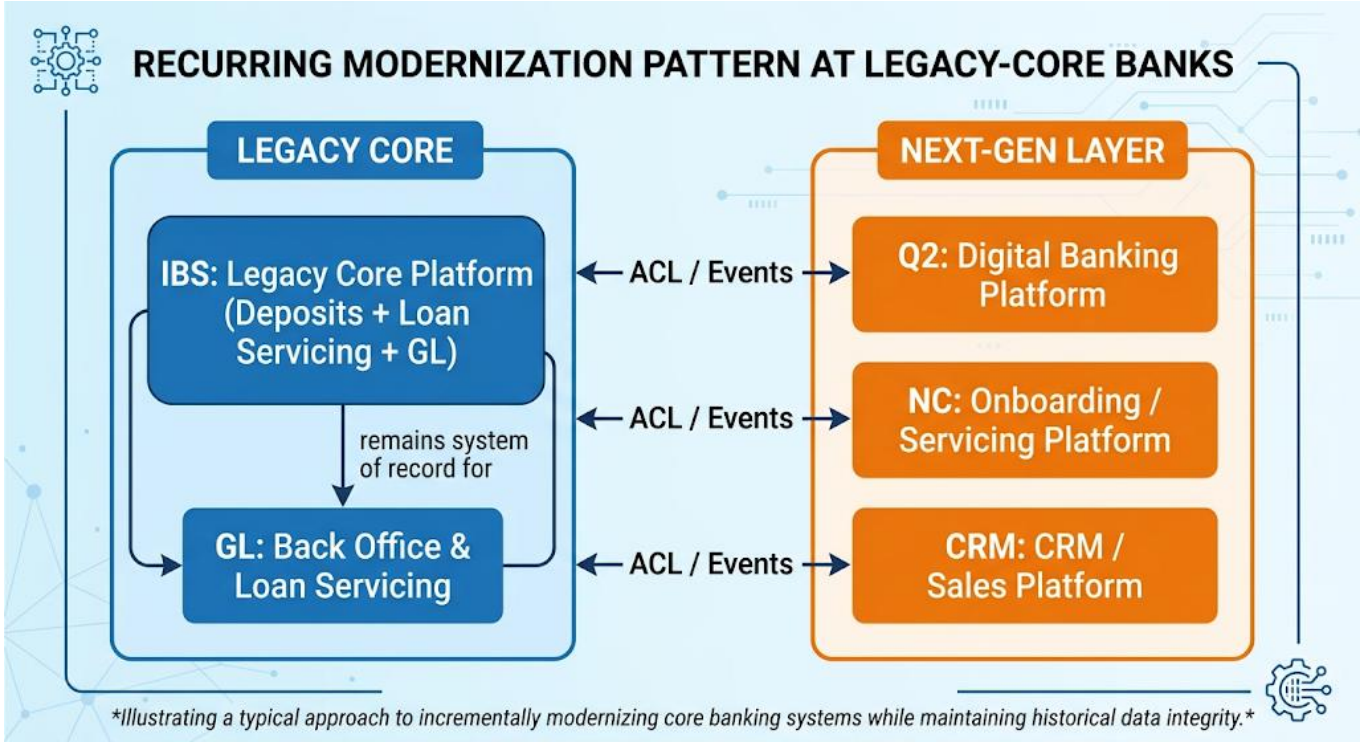
Step 7) Core Hollowing-Out Pattern

Keep the legacy core as system of record for high-risk back-office processing and areas with high switching cost.

Incrementally migrate customer-facing and rapidly changing domains.

Use anti-corruption layers and explicit event contracts so legacy data shapes do not leak into new platforms.

Narrow the legacy core's scope one well-chosen domain boundary at a time.



Rather than a "big bang" core replacement , which carries enormous regulatory, operational, and change-management risk , the bank:

1. Keeps the legacy core as the system of record for back-office processing and loan servicing, where switching cost and regulatory risk are highest
2. Incrementally migrates customer-facing and rapidly-changing domains (digital banking, onboarding, sales/CRM) to modern, independently-deployable platforms
3. Connects the two through an anti-corruption layer and an explicit event contract, so the legacy core's internal data shapes never leak into the new platforms
4. Evaluates the core vendor's own modernization roadmap opportunistically, rather than betting the whole program on either the full status quo or a full rip-and-replace

This is the essence of "hollowing out" a core: the legacy system doesn't disappear on day one, but its *scope of responsibility* is deliberately and continuously narrowed, one well-chosen domain boundary at a time, until only the parts genuinely best served by that platform remain.

Step 8) Common Pitfalls

- Boundary by org chart: preserves today's organizational dysfunction.
- Boundary by database table: simply relocates the legacy model.
- One shared kernel for everything: creates a canonical model that satisfies nobody well.
- No published contract: recreates point-to-point coupling.
- Sequencing by ease instead of business value: produces little visible business benefit.

Step 8) Maturity Checklist

A bank can gauge its own progress with a few honest questions:.

- Can we name the 8 to 12 domains that make up the bank in plain business language?
- Does each domain have an authoritative owner for its data?
- Does every cross-domain interaction have an explicit, versioned API or event contract?
- Have we identified which domain to extract first and why?
- Does team structure support the target domain map?

Conclusion

Core modernization programs don't fail because banks chose the wrong technology , they fail because the technology was asked to encode boundaries that were never actually agreed on. Finding domain boundaries is slower and less glamorous than picking a target architecture diagram, but it's the step that determines whether every subsequent investment compounds or gets undone by the next reorg.

- The technology should encode boundaries that the business has already understood and agreed upon.
- Capability taxonomies such as BIAN, Event Storming, context mapping, and disciplined sequencing provide a repeatable way to discover and validate boundaries.
- The objective is not a perfect diagram. It is a sequence of independently valuable modernization moves.

Prepared by Placida Fernando